ASSIGNMENT 2 – CO2

Control Flow Constructs, String Operations and Functions in Python

SUBMITTED BY:- M SANTHOSH KUMAR(192412268)

Q1 – Decisions with Many Boundaries (6 Marks)

Problem analysis and predictions

Subsidy rules: households consuming 100 units or less are eligible; from 101 to 200 units, a partial subsidy is given only when the household has a ration card; above 200 units, no subsidy is given unless a household member is older than 80. The brief does not specify the amount or exact type of subsidy for the over-200 exception, so this solution labels that case as 'SPECIAL REVIEW / ELIGIBLE' rather than inventing an amount.

Household	Units	Ration card?	Member >80?	Predicted decision
A	100	No	No	Full subsidy
B	101	Yes	No	Partial subsidy
C	101	No	No	No subsidy
D	200	Yes	No	Partial subsidy
E	201	Yes	No	No subsidy
F	201	No	Yes	Special eligibility exception
G	75	Yes	Yes	Full subsidy
H	200	No	Yes	No subsidy under 101–200 rule

Implementation 1: separate if statements

```
def subsidy_separate(units, ration_card, member_over_80):
    """Demonstrate separate if statements; overlapping decisions can occur."""
    results = []
    if units <= 100:
        results.append("Full subsidy")
    if 101 <= units <= 200 and ration_card:
        results.append("Partial subsidy")
    if units > 200 and member_over_80:
        results.append("Special eligibility exception")
    if not results:
        results.append("No subsidy")
    return results
```

Implementation 2: mutually exclusive if–elif–else

```
def subsidy_exclusive(units, ration_card, member_over_80):
    """Return one decision using mutually exclusive branches."""
    if units <= 100:
        return "Full subsidy"
    elif 101 <= units <= 200:
        return "Partial subsidy" if ration_card else "No subsidy"
    else:
        return "Special eligibility exception" if member_over_80 else "No subsidy"

cases = [
    (100, False, False), (101, True, False), (101, False, False),
```

```
    (200, True, False), (201, True, False), (201, False, True),
    (75, True, True), (200, False, True)
]
for c in cases:
    print(c, "=>", subsidy_separate(*c), "| exclusive:", subsidy_exclusive(*c))
```

Separate if statements are independent, so more than one block can run if conditions overlap. In this particular set of mutually exclusive unit ranges, overlap is limited, but separate statements can still produce multiple results when rules are written too broadly. if–elif–else selects at most one branch. Combined Boolean conditions are suitable when each condition remains short; nested if statements are useful when the second test only matters after the first is true. For this rule, the if–elif–else structure makes the consumption slabs clear.

Alternative concept and complexity

```
def subsidy_match(units, ration_card, member_over_80):
    match units:
        case u if u <= 100:
            return "Full subsidy"
        case u if 101 <= u <= 200:
            return "Partial subsidy" if ration_card else "No subsidy"
        case _:
            return "Special eligibility exception" if member_over_80 else "No subsidy"
```

A match–case implementation with guards is available in Python 3.10+. A list of upper-limit/decision pairs can be searched in order, stopping at the first matching slab. For k slabs, a sequential lookup takes O(k) worst-case time and O(k) space for the table. if–elif and guarded match–case have O(1) time for a fixed number of branches and O(1) extra space. A data-driven table is easier to update when many slabs change frequently.

Q2 – Loops that Count Attempts (4 Marks)

A blank OTP must not count as an attempt. A wrong nonblank OTP increments the attempt counter. A correct OTP exits using break. The loop else clause runs only when the loop finishes normally, without break, so it can lock the account after all allowed attempts.

while loop

```
def verify_otp_while(correct_otp):
    attempts = 0
    while attempts < 3:
        entered = input("Enter OTP: ").strip()
        if entered == "":
            print("Blank entry; try again. Attempt not counted.")
            continue
        attempts += 1
        if entered == correct_otp:
            print("OTP verified")
            break
        print("Incorrect OTP")
    else:
        print("Account locked for 30 minutes")
```

for loop with range(3)

```
def verify_otp_for(correct_otp):
    for attempt in range(3):
        entered = input("Enter OTP: ").strip()
        if entered == "":
            print("Blank entry; try again. Attempt not counted.")
            # The for loop still consumes an iteration, so use a retry counter
            # or a while loop when blanks must not consume one of three attempts.
            continue
        if entered == correct_otp:
            print("OTP verified")
            break
```

```
            print("Incorrect OTP")
        else:
            print("Account locked for 30 minutes")
```

Important: the simple for-range version above consumes an iteration on a blank entry, so it does not fully satisfy the requirement. A correct for-based design needs a separate counted-attempt variable and a while loop around each counted attempt, or should prefer while for this exact rule. The while version naturally counts only nonblank entries. Updating the counter before checking for a blank is a common bug; failing to update it after a wrong OTP can create an infinite loop.

Improved for-style counted attempts

```
def verify_otp_for_counted(correct_otp):
    wrong_attempts = 0
    for _ in iter(int, 1):  # deliberate indefinite iterator; stop with break
        entered = input("Enter OTP: ").strip()
        if entered == "":
            print("Blank entry; not counted")
            continue
        wrong_attempts += 1
        if entered == correct_otp:
            print("OTP verified")
            break
        if wrong_attempts == 3:
            print("Account locked for 30 minutes")
            break
```

The iter(int, 1) idiom is less readable than while for this use case; the while loop is recommended. break exits immediately, continue skips the remaining body and starts the next iteration, and pass is a no-operation placeholder. The loop else runs when no break occurs. If a correct security answer allows two additional attempts, set the maximum to 5 only after verifying the security answer; otherwise keep the limit at 3.

Recursive alternative and complexity

```
def verify_recursive(correct_otp, attempts_left=3):
    if attempts_left == 0:
        print("Account locked for 30 minutes")
        return False
    entered = input("Enter OTP: ").strip()
    if not entered:
        print("Blank entry; not counted")
        return verify_recursive(correct_otp, attempts_left)
    if entered == correct_otp:
        print("OTP verified")
        return True
    return verify_recursive(correct_otp, attempts_left - 1)
```

For a attempts, all approaches take O(a) time. A loop uses O(1) auxiliary space, while recursion uses O(a) call-stack space and is less appropriate for a simple retry counter.

Q3 – Vehicle Registration Number Validator (6 Marks)

Accepted normalized pattern: two letters, two digits, one or two letters, and four digits. Total length is 9 or 10. Spaces and hyphens are removed and letters are converted to uppercase.

```
def normalise(text):
    """Remove spaces and hyphens, then uppercase the registration."""
    return text.strip().replace(" ", "").replace("-", "").upper()

def is_valid(reg):
    """Validate the state, district, series and final four digits."""
    if len(reg) not in (9, 10):
        return False
    if not reg[0:2].isalpha() or not reg[2:4].isdigit():
```

```
        return False
    series_len = len(reg) - 8
    if series_len not in (1, 2):
        return False
    if not reg[4:4 + series_len].isalpha():
        return False
    if not reg[4 + series_len:].isdigit():
        return False
    return True

def format_registration(reg):
    """Format a valid normalized registration for a receipt."""
    if not is_valid(reg):
        raise ValueError("Invalid registration")
    series_len = len(reg) - 8
    return f"{reg[:2]} {reg[2:4]} {reg[4:4+series_len]} {reg[4+series_len:]}"

tests = ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234",
         "TN 22 1234 AB", "ap 39 c 9999", "MH 12 DE 1433"]
for raw in tests:
    reg = normalise(raw)
    print(f"{raw:15} -> {reg:11} {'valid' if is_valid(reg) else 'INVALID'}")
```

Input	Normalized	Predicted result
tn 09 ab 1234	TN09AB1234	Valid
KA-01-M-0007	KA01M0007	Valid
TN9AB1234	TN9AB1234	Invalid: district has one digit
TN 22 1234 AB	TN221234AB	Invalid: series appears after final digits
ap 39 c 9999	AP39C9999	Valid
MH 12 DE 1433	MH12DE1433	Valid

Strings are immutable: a string operation creates a new string instead of changing the existing object. text.upper() returns an uppercase string; it does not mutate text, so assign the result if it is needed. For a 9-character string s, s[-4:] returns the final four characters and s[4:-4] returns one character (index 4). For a 10-character string, s[-4:] again returns the last four characters and s[4:-4] returns two characters.

A regular expression alternative is re.fullmatch(r'[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}', reg). Both approaches take O(L) time for string length L. Regex is compact for formal patterns; slicing makes the individual fields and their rules more explicit.

Q4 – Building Long Text: + or join()? (6 Marks)

```
import time

items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]

def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text

def build_join(parts):
    return "".join(p + ";" for p in parts)

def benchmark(parts):
    start = time.perf_counter()
    a = build_plus(parts)
    t_plus = time.perf_counter() - start
    start = time.perf_counter()
    b = build_join(parts)
    t_join = time.perf_counter() - start
```

```
    print("same text:", a == b, "| length:", len(b))
    print(f"+= time: {t_plus:.4f} s | join time: {t_join:.4f} s")
    return t_plus, t_join

for n in (1_000, 10_000, 200_000):
    print("N =", n)
    benchmark(items[:n])
```

Expected text equality is True. For the supplied 200,000-item data, the expected final length is 3,048,890 characters. Timing depends on the computer and Python build; run the benchmark three times per size and calculate the average rather than copying a made-up timing.

Strings are immutable, so repeated += can require creating larger strings and copying existing characters; this can approach $O(n^2)$ copied characters in an unfavourable implementation. CPython can optimise some += patterns, so measured results may be closer than the theoretical warning suggests. join() collects the parts and constructs the final string efficiently in O(total output length) time, at the cost of memory for the parts and final output.

Alternatives

```
from io import StringIO

def build_stringio(parts):
    buffer = StringIO()
    for p in parts:
        buffer.write(p)
        buffer.write(";")
    return buffer.getvalue()

def write_to_file(parts, filename):
    with open(filename, "w", encoding="utf-8") as f:
        for p in parts:
            f.write(p)
            f.write(";")
```

StringIO is useful when output must be assembled incrementally and then retrieved as one string. Writing directly to a file avoids holding the entire final statement in memory, which is useful for very large outputs. join() is appropriate when the complete statement is required as a string.

Q5 – Functions, Parameters and Scope (7 Marks)

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order

print(add_item_buggy("Tea"), add_item_buggy("Vada"))
print(add_item("Tea"), add_item("Vada"))
print("buggy defaults:", add_item_buggy.__defaults__)
```

The default list in add_item_buggy is created once when the function is defined, so calls share the same list. The None pattern creates a fresh list on each call unless the caller provides an existing list. Expected output for the two print lines is ['Tea', 'Vada'] ['Tea', 'Vada'] and ['Tea'] ['Vada']. The defaults tuple shows the shared list object used by the buggy function.

Parameters and multiple return values

```
def price_item(name, quantity, unit_price=0):
    """Return subtotal and GST for one item."""
    subtotal = quantity * unit_price
```

```
    gst = subtotal * 0.05
    return subtotal, gst

print(price_item("Tea", 2, unit_price=15))
```

Positional parameters are concise when the order is obvious; keyword parameters improve clarity; default parameters make optional values convenient. Returning subtotal, GST as a tuple allows the caller to unpack both values.

Local, global and nonlocal examples

```
count = 0

def increment_global():
    global count
    count += 1

def outer():
    message = "outer"
    def inner():
        nonlocal message
        message = "updated"
    inner()
    return message

def local_example():
    value = "local"
    return value

increment_global()
print(count, outer(), local_example())
```

A local name belongs to a function call. A global name is defined at module scope; assigning to it inside a function requires global. A nonlocal name refers to a binding in an enclosing function scope. Mutating shared global state is usually avoided because it makes testing and debugging harder; pass values as parameters and return updated results instead.

Three-customer independence test

```
customers = [
    add_item("Tea"),
    add_item("Vada"),
    add_item("Coffee")
]
for order in customers:
    print(order)
```

Each call to add_item without an order argument returns a separate list, so customers' orders remain independent. If an order is explicitly supplied, the function appends to that supplied list. Time per append is amortized O(1); each customer's list requires O(m) space for m items.

Q6 – Recursion and Its Limits (8 Marks)

Let ways(n) count the ways to climb n steps taking either one or two steps. Base cases: ways(0) = 1 and ways(1) = 1. Recursive case: ways(n) = ways(n−1) + ways(n−2). This is the Fibonacci sequence shifted by one position.

```
from functools import lru_cache

calls = 0

def ways_naive(n):
    """Count ways using plain recursion."""
    global calls
    calls += 1
    if n <= 1:
        return 1
```

```
    return ways_naive(n - 1) + ways_naive(n - 2)

@lru_cache(maxsize=None)
def ways_memo(n):
    """Count ways using memoisation."""
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)

for n in range(1, 7):
    print(n, ways_memo(n))

calls = 0
print("naive:", ways_naive(25), "calls =", calls)
calls = 0
ways_memo.cache_clear()
print("memo :", ways_memo(25), "calls =", calls)
```

n	Ways by hand / expected
1	1
2	2
3	3
4	5
5	8
6	13

For n = 5, the plain call tree starts ways(5) → ways(4) + ways(3); ways(4) → ways(3) + ways(2); ways(3) is computed repeatedly in both branches. This repeated work makes plain recursion exponential, $O(\varphi^n)$, where φ is the golden ratio. Memoisation computes each distinct n once, taking O(n) time and O(n) cache/stack space.

Count calls and test n = 1,000

```
for n in (10, 20, 25, 30):
    calls = 0
    ways_naive(n)
    naive_calls = calls
    calls = 0
    ways_memo.cache_clear()
    ways_memo(n)
    memo_calls = calls
    print(n, naive_calls, memo_calls)

import sys
print("recursion limit:", sys.getrecursionlimit())
try:
    ways_memo.cache_clear()
    print(ways_memo(1000))
except RecursionError as e:
    print("RecursionError:", e)
```

The plain recursive version grows rapidly; memoisation has approximately n+1 uncached function calls when the cache is cleared before the run. The exact call counts should be recorded from actual execution. A recursive memoised version may raise RecursionError at n = 1,000 because each recursive call consumes stack depth and Python has a recursion limit. Raising the limit can be unsafe if set excessively high.

Iterative O(1)-space solution

```
def ways_iterative(n):
    """Count ways using only the previous two values."""
    if n <= 1:
        return 1
    previous2, previous1 = 1, 1
```

```
    for _ in range(2, n + 1):
        current = previous1 + previous2
        previous2, previous1 = previous1, current
    return previous1

print("iterative n=1000:", ways_iterative(1000))
```

The iterative solution takes O(n) arithmetic steps and O(1) auxiliary variables, although the integer itself grows in number of digits as n increases. It avoids recursion depth issues and is the recommended deployment version for n up to 1,000. The memoised recursive version is helpful for demonstrating recursion and repeated-subproblem elimination.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected program: Q6 staircase ways. It has a clear, measurable scalability limitation: plain recursion repeats the same subproblems many times. This is supported by the expected count for n = 25 in the assignment brief: 242,785 calls for the naive version versus 26 calls for memoisation.

A. Diagnose the bottleneck – 2 marks

The main bottleneck is repeated evaluation of ways(n−1) and ways(n−2) across overlapping branches. The function-call count grows exponentially with n in the naive implementation. The bottleneck is not input/output; it is duplicate recursive computation.

B. Define the design objective – 2 marks

Objective: compute the number of ways for n = 1,000 without RecursionError, with O(n) loop iterations and O(1) auxiliary variables. Validate correctness by comparing the redesigned result with memoisation for small n and measure runtime and operation counts.

C. Design the revised solution – 4 marks

```
PSEUDOCODE
FUNCTION ways_iterative(n)
    IF n <= 1
        RETURN 1
    SET previous2 = 1
    SET previous1 = 1
    FOR step FROM 2 TO n
        SET current = previous1 + previous2
        SET previous2 = previous1
        SET previous1 = current
    RETURN previous1
```

The redesign stores only the previous two sequence values and computes each next value once. It trades the simplicity of a recursive mathematical definition for an iterative loop. It needs O(1) auxiliary variables and does not use the Python call stack recursively.

D. Predict before implementing – 1 mark

Prediction: for n = 25, the iterative version will return the same result, 121393, while performing a number of loop iterations proportional to n rather than 242,785 recursive calls. For n = 1,000, the iterative version should finish without RecursionError.

E. Implement and validate – 2 marks

```
import time

for n in (10, 20, 25, 30, 1000):
    start = time.perf_counter()
    result = ways_iterative(n)
    elapsed = time.perf_counter() - start
    print(f"n={n:4} result_digits={len(str(result)):4} time={elapsed:.6f}s")

# Cross-check the first several cases
```

```
ways_memo.cache_clear()
for n in range(1, 31):
    assert ways_iterative(n) == ways_memo(n)
print("Validation passed for n = 1..30")
```

Version	Time complexity	Auxiliary space	Expected behaviour
Naive recursion	$O(\varphi^n)$	O(n) stack	Repeated work; becomes slow quickly
Memoised recursion	O(n)	O(n) cache and stack	Avoids repeated work but can hit recursion limit
Iterative two-value	O(n)	O(1) auxiliary variables	No recursive depth; recommended for n=1,000

Record actual times and results from your machine. Do not invent timing values. Run each version several times where practical and report the median or average, explaining that machine load affects measured time.

F. Defend the final design – 2 marks

The iterative design meets the objective because it avoids repeated work and recursive depth while retaining linear time. Memoisation is still useful when the same subproblems are requested repeatedly or when the recursive formulation is pedagogically important. Plain recursion remains suitable only for small n where its clarity outweighs its exponential cost.

Overall Conclusion

The assignment demonstrates structured decisions, retry loops, string normalization and validation, efficient text construction, safe function defaults, scope, recursion and memoisation. The final recommendation is to use clear mutually exclusive branches for subsidy slabs, a while loop for counted OTP attempts, explicit string validation, join() or streaming for large text, None for mutable defaults, and an iterative dynamic-programming approach for the staircase problem at n = 1,000.

Submission checklist

- Fill in name, register number and GitHub repository link.
- Create repository CSA08-<RegisterNumber>-Assignment2.
- Store each question in a separate Q1–Q7 or Task01–Task07 folder.
- Run every program and add output screenshots.
- For Q4, record three runs and average timings for 1,000, 10,000 and 200,000 lines.
- For Q6/Q7, record raw function-call counts and actual timings for n = 10, 20, 25 and 30, and test n = 1,000 iteratively.
- Add the actual Python version and requirements.txt to README.md; make progressive commits for each task.